



# Complete Interview Preparation Guide

## Technology Deep-Dive · Architecture Rationale · 100+ Q&A;

|           |                 |                 |             |        |          |
|-----------|-----------------|-----------------|-------------|--------|----------|
| React 19  | React Router v7 | React Hook Form | TailwindCSS | Vite 8 |          |
| Node.js   | Express         | MongoDB         | Mongoose    | JWT    | bcryptjs |
| Gemini AI | Axios           | Archiver        | Nodemailer  | Vercel | Render   |

[buildfolio-one.vercel.app](https://buildfolio-one.vercel.app) · [github.com/Mdhummad/buildfolio](https://github.com/Mdhummad/buildfolio)

This guide covers every technology used, architectural decisions, real code from your project, and interview questions with model answers.

---

# TABLE OF CONTENTS

---

|    |                                                         |
|----|---------------------------------------------------------|
| 01 | Project Architecture & Folder Structure                 |
| 02 | React 19 — Component Model, Hooks, JSX                  |
| 03 | React Hook Form — Form State & Validation               |
| 04 | React Router DOM v7 — SPA Navigation                    |
| 05 | TailwindCSS v3 + Glassmorphism CSS Design System        |
| 06 | Vite 8 — Build Tool & Dev Server                        |
| 07 | Node.js + Express — REST API Backend                    |
| 08 | MongoDB Atlas + Mongoose — Database Layer               |
| 09 | JWT Authentication — Stateless Auth System              |
| 10 | bcryptjs — Password Hashing                             |
| 11 | Axios — HTTP Client + Interceptors                      |
| 12 | React Context API — Global State Management             |
| 13 | Google Gemini AI — LLM Integration & Prompt Engineering |
| 14 | Archiver — ZIP File Generation & Streaming              |
| 15 | Nodemailer — Transactional Email (Password Reset)       |
| 16 | Deployment — Vercel · Render · MongoDB Atlas            |
| 17 | Security — CORS, Env Vars, Hashing, Token Expiry        |
| 18 | Master Interview Q&A; — 100 Questions Across All Topics |
| 19 | How to Talk About This Project in an Interview          |

Buildfolio is a **monorepo** — frontend and backend live in the same Git repository. One git push auto-deploys both. Vercel handles the React frontend; Render handles the Express backend.

```
buildfolio/
  ■■■ backend/ ← Express API (deployed to Render)
  ■ ■■■ middleware/auth.js ← JWT verification (reused by all protected routes)
  ■ ■■■ models/User.js ← MongoDB User schema
  ■ ■■■ models/Portfolio.js ← MongoDB Portfolio schema
  ■ ■■■ routes/auth.js ← /api/auth (register, login, forgot/reset password)
  ■ ■■■ routes/portfolio.js ← /api/portfolio (GET, PUT)
  ■ ■■■ routes/export.js ← /api/export (ZIP generation + download)
  ■ ■■■ routes/gemini.js ← /api/gemini (AI resume optimization)
  ■ ■■■ server.js ← Entry point: CORS, DB connect, route mounting
  ■
  ■■■ src/ ← React frontend (deployed to Vercel)
  ■ ■■■ components/ ← Shared UI: NavBar, AuthModal
  ■ ■■■ context/ ← AuthContext, PortfolioContext
  ■ ■■■ pages/ ← LandingPage, BuilderPage, TemplatePage, PreviewPage
  ■ ■■■ portfolio-templates/ ← 5 self-contained template folders + index.js registry
  ■ ■■■ utils/api.js ← Single Axios instance (base URL + interceptors)
  ■ ■■■ index.css ← Design system (glassmorphism, glows, gradients)
  ■
  ■■■ vercel.json ← SPA rewrite: all routes -> index.html
  ■■■ render.yaml ← Backend deploy config
```

## Why Monorepo?

|                                                    |                                                                                                  |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;Single push deploys both&lt;/b&gt;</b> | One git push to main → Vercel rebuilds frontend + Render rebuilds backend simultaneously.        |
| <b>&lt;b&gt;Shared context&lt;/b&gt;</b>           | Frontend devs can see backend routes, models, and data shapes in the same codebase.              |
| <b>&lt;b&gt;One PR for full features&lt;/b&gt;</b> | A feature that touches both frontend and backend is one pull request — easier to review.         |
| <b>&lt;b&gt;Platform intelligence&lt;/b&gt;</b>    | Vercel only rebuilds if files in /src change; Render only if /backend changes. No wasted builds. |

## Key Architectural Decisions

|                                                  |                                                                                                           |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;One Axios instance&lt;/b&gt;</b>     | utils/api.js configures base URL, JWT header, and 401 logout once. Every component just calls api.g       |
| <b>&lt;b&gt;Template registry&lt;/b&gt;</b>      | portfolio-templates/index.js exports all 5 templates. Both TemplatePage and PreviewPage import fro        |
| <b>&lt;b&gt;Context not Redux&lt;/b&gt;</b>      | Only 2 global states needed (auth + portfolio). Context API is sufficient — no Redux boilerplate need     |
| <b>&lt;b&gt;Route-per-domain&lt;/b&gt;</b>       | auth.js, portfolio.js, export.js, gemini.js are each self-contained. Drop a feature by deleting one file. |
| <b>&lt;b&gt;No disk writes for ZIP&lt;/b&gt;</b> | Archiver streams ZIP directly to HTTP response — no temp files created on the server.                     |

### O: Whv did vou choose a monorepo structure for Buildfolio?

A: Having frontend and backend in one repo means a single git push deploys both. Both Vercel and Render are smart enough to only rebuild their relevant folder when code changes. It also makes the codebase easier to reason about — you can trace a full feature from the React component through the API route to the database model without switching repositories.

**O: What is the role of `utils/api.is` and why is it important?**

A: It's a single configured Axios instance shared by the entire frontend. The base URL (dev vs production), JWT token attachment, and automatic logout on 401 responses are all configured once in this file. Without it, every component would need to manage headers, base URLs, and error handling — leading to duplication and inconsistency.

React is a **JavaScript library for building user interfaces** using a component-based architecture. Buildfolio uses React 19 — the latest major version, which introduced concurrent features, improved server component support, and performance optimisations.

## Core Concepts

|                                            |                                                                                                                 |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;Component&lt;/b&gt;</b>        | A reusable, self-contained piece of UI. Can be a button, a form step, or an entire page. Returns JSX.           |
| <b>&lt;b&gt;JSX&lt;/b&gt;</b>              | JavaScript XML — HTML-like syntax inside JS files. Compiled by Vite/Babel to <code>React.createElement</code> . |
| <b>&lt;b&gt;Props&lt;/b&gt;</b>            | Data passed from parent to child. Read-only. Like function arguments for components.                            |
| <b>&lt;b&gt;State (useState)&lt;/b&gt;</b> | Data local to a component. When state changes, React re-renders only that component and its children.           |
| <b>&lt;b&gt;useEffect&lt;/b&gt;</b>        | Runs side effects after render: API calls, event listeners, timers. Dependency array controls when it runs.     |
| <b>&lt;b&gt;Virtual DOM&lt;/b&gt;</b>      | React maintains a lightweight copy of the DOM. On state change, it diffs the new vs old virtual DOM.            |
| <b>&lt;b&gt;Reconciliation&lt;/b&gt;</b>   | The algorithm React uses to compare virtual DOM trees and determine the minimal set of real DOM updates.        |
| <b>&lt;b&gt;Key prop&lt;/b&gt;</b>         | A unique identifier on list items. Tells React which item changed/added/removed when re-rendering.              |

## How Buildfolio Uses React

- BuilderPage uses `useState` to track `currentStep` (1–6) and conditionally renders each step component
- `useEffect` loads saved portfolio data from MongoDB after login via the `PortfolioContext`
- Dynamic experience/project lists use `Array.map()` with unique `key` props
- Profile photo upload converts the file to base64 using `FileReader` and stores it in state
- The `PreviewPage` re-renders the selected template live whenever `portfolioData` changes

```
// BuilderPage.jsx — step navigation with useState
import { useState, useEffect } from "react";
import { usePortfolio } from "../context/PortfolioContext";

function BuilderPage() {
  const [currentStep, setCurrentStep] = useState(1);
  const { portfolioData, loadFromServer } = usePortfolio();

  // Load saved data when component mounts
  useEffect(() => {
    loadFromServer();
  }, []); // empty array = run once on mount

  return (
    <div>
      {currentStep === 1 && <PersonalStep />}
      {currentStep === 2 && <ExperienceStep />}
      {/* ... */}
      <button onClick={() => setCurrentStep(s => Math.max(1, s-1))}>Back</button>
      <button onClick={() => setCurrentStep(s => Math.min(6, s+1))}>Next</button>
    </div>
  );
}
```

```

}

// Dynamic list rendering with key prop
function ExperienceList({ experiences }) {
  return (
    <ul>
      {experiences.map((exp, index) => (
        <li key={exp.id || index}> {/* key must be unique */}
        <b>{exp.role}</b> at {exp.company}
      </li>
      )}}
    </ul>
  );
}

```

## Interview Questions — React

### O: What is the Virtual DOM and why does React use it?

A: The Virtual DOM is a lightweight JavaScript representation of the real DOM. When state changes, React creates a new virtual DOM tree, diffs it against the previous one (reconciliation), and applies only the changed parts to the real DOM. Direct DOM manipulation is expensive — React batches and minimises these operations for performance.

### O: Explain the `useEffect` dependency array.

A: The dependency array controls when the effect re-runs. An empty array `[]` means run once on mount (like `componentDidMount`). `[value]` means re-run whenever value changes. No array at all means run after every render. In Buildfolio, `useEffect(loadFromServer, [])` loads portfolio data once when `BuilderPage` mounts.

### O: Why do we need `key` props on lists?

A: React uses keys to identify which items in a list changed, were added, or were removed. Without keys, React falls back to re-rendering the entire list on any change. In Buildfolio's experience/project lists, each entry gets a unique key so React can update only the modified entry.

### O: What is the difference between controlled and uncontrolled components?

A: A controlled component has its value driven by React state — `onChange` updates the state, value reads from the state. An uncontrolled component stores its value in the DOM and you use a ref to read it. React Hook Form uses uncontrolled inputs with refs, which is why it's more performant — no re-render on every keystroke.

### O: What's new or notable in React 19?

A: React 19 introduced the React Compiler (automatic memoisation — eliminates most manual `useMemo/useCallback`), first-class support for async transitions (`useTransition` for non-blocking state updates), and improved error handling. For Buildfolio, React 19 means better out-of-the-box performance without manual optimisation.

React Hook Form (RHF) is a performant form library that uses **uncontrolled inputs** and refs instead of controlled state. This means the form doesn't re-render on every keystroke — only on validation triggers and submission.

|                                       |                                                                                                           |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>&lt;register()&lt;/b&gt;</b>       | Connects an HTML input to the form. Replaces manual value + onChange. Accepts validation rules.           |
| <b>&lt;handleSubmit()&lt;/b&gt;</b>   | Wrapper for your submit function. Runs all validation first; only calls your function if all fields pass. |
| <b>&lt;formState.errors&lt;/b&gt;</b> | Object containing validation errors per field. Use to show error messages under inputs.                   |
| <b>&lt;watch()&lt;/b&gt;</b>          | Subscribes to a field's value in real time. Used for conditional fields (e.g. 'Present' checkbox).        |
| <b>&lt;setValue()&lt;/b&gt;</b>       | Programmatically set a field's value. Used when loading saved data into the form.                         |
| <b>&lt;reset()&lt;/b&gt;</b>          | Resets all fields to default values or a provided object. Used after successful save.                     |
| <b>&lt;useFieldArray()&lt;/b&gt;</b>  | Manages dynamic arrays of fields. append(), remove(), move() for Experience/Project lists.                |
| <b>&lt;Controller&lt;/b&gt;</b>       | Wrapper for third-party controlled inputs (like a custom tag input) that don't work with register().      |

```
// SkillsStep.jsx — tag-based skill input with validation
import { useForm, Controller } from "react-hook-form";

function SkillsStep() {
  const { register, handleSubmit, formState: { errors }, setValue, watch } = useForm({
    defaultValues: {
      technical: [],
      languages: []
    }
  });

  const onSubmit = (data) => {
    // Only runs if all validation passes
    console.log(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input
        {...register("technical", {
          required: "Add at least one skill",
          minLength: { value: 2, message: "Too short" }
        })}
        placeholder="e.g. React"
        {errors.technical && <span>{errors.technical.message}</span>}
      />

      <button type="submit">Next Step</button>
    </form>
  );
}

// useFieldArray — dynamic experience list
import { useFieldArray } from "react-hook-form";

const { fields, append, remove } = useFieldArray({ control, name: "experiences" });
```



```
// Append a new blank experience entry
<button onClick={() => append({ company:"", role:"", startDate:"", endDate:"" })}>
+ Add Experience
</button>

// Render each entry
{fields.map((field, i) => (
<div key={field.id}>
<input {...register(`experiences.${i}.company`, { required: true })} />
<input {...register(`experiences.${i}.role`)} />
<button onClick={() => remove(i)}>Remove</button>
</div>
))}
```

### **O: Why did you choose React Hook Form over managing form state manually with useState?**

A: Manual state means calling `setState` on every keystroke, which triggers a re-render of the entire form — very inefficient for a 6-step wizard with dozens of fields. React Hook Form uses uncontrolled inputs and refs, so the component only re-renders on validation and submission. It also provides built-in validation rules, error state, and `useFieldArray` for dynamic lists — all things I would have needed to build manually.

### **O: How does `useFieldArray` work? Where do you use it in Buildfolio?**

A: `useFieldArray` manages an array of form fields that users can dynamically add or remove. It gives you a fields array to render, an `append()` function to add entries, and `remove(index)` to delete them. I use it for the Experience step (unlimited work history entries), Education step, and Projects step. Each entry gets a stable `field.id` as the React key.

React Router is the standard routing library for React SPAs. It intercepts browser navigation events and renders the correct component without a server round-trip — making navigation instant.

|                                      |                                                                                                                              |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>&lt;SPA (Single Page App)&gt;</b> | One HTML file loads once. JavaScript handles all navigation. No full page reloads. URL changes but the page content changes. |
| <b>&lt;BrowserRouter&gt;</b>         | Uses the HTML5 History API to keep UI in sync with the URL. Wraps the entire app.                                            |
| <b>&lt;Routes + Route&gt;</b>        | Declarative mapping of URL paths to components. <Route path='/builder' element={<BuilderPage />} />                          |
| <b>&lt;Link&gt;</b>                  | Like <a href> but prevents page reload. Updates URL and renders the matched component.                                       |
| <b>useNavigate()</b>                 | Hook to programmatically change the route. Used after wizard completion to go to /templates.                                 |
| <b>useLocation()</b>                 | Returns current URL info. Used to highlight the active nav item in Navbar.                                                   |
| <b>useParams()</b>                   | Extracts dynamic URL segments. e.g. /portfolio/:id extracts id.                                                              |
| <b>&lt;Nested Routes&gt;</b>         | Routes can be nested for layouts that share a common wrapper component.                                                      |

```
// App.jsx – full routing setup
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
    <Navbar />
    <Routes>
    <Route path="/" element={<LandingPage />} />
    <Route path="/builder" element={<ProtectedRoute><BuilderPage /></ProtectedRoute>} />
    <Route path="/templates" element={<ProtectedRoute><TemplatePage /></ProtectedRoute>} />
    <Route path="/preview" element={<ProtectedRoute><PreviewPage /></ProtectedRoute>} />
    <Route path="*" element={<Navigate to="/" />} /> {/* catch-all */}
    </Routes>
    </BrowserRouter>
  );
}

// Protected Route pattern – redirect to / if not logged in
function ProtectedRoute({ children }) {
  const { isAuthenticated } = useAuth();
  return isAuthenticated ? children : <Navigate to="/" replace />;
}

// Programmatic navigation after completing wizard
import { useNavigate } from "react-router-dom";
function FinalStep() {
  const navigate = useNavigate();
  const handleFinish = () => navigate("/templates");
  return <button onClick={handleFinish}>Choose Template</button>;
}

// vercel.json – required for SPA deployment on Vercel
// Without this, refreshing /builder returns a 404 from Vercel's CDN
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

**O: Why does Buildfolio need a vercel.json with a rewrite rule?**

A: Vercel's CDN serves static files. When a user visits /builder directly or refreshes the page, Vercel looks for a file at that path — which doesn't exist (there's only index.html). The rewrite rule tells Vercel to serve index.html for any path, and React Router then reads the URL client-side to render the correct component. Without this, any direct URL or refresh that isn't / would return a 404.

**O: What is the difference between useNavigate and a Link component?**

A: Link is used in JSX for navigation that users trigger by clicking — like a nav menu. useNavigate is used programmatically inside event handlers or effects — like redirecting after a form submission or login. In Buildfolio, after the user completes the builder wizard, useNavigate('/templates') sends them to the template selection page automatically.

# TailwindCSS Core Concepts

|                                               |                                                                                                      |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;Utility classes&lt;/b&gt;</b>     | Single-purpose classes: p-4 (padding), text-lg (font size), bg-cyan-500 (background). Compose them   |
| <b>&lt;b&gt;Responsive prefixes&lt;/b&gt;</b> | md:text-lg (applies at medium breakpoints), lg:grid-cols-3. Mobile-first by default.                 |
| <b>&lt;b&gt;State variants&lt;/b&gt;</b>      | hover:bg-cyan-400, focus:ring-2, active:scale-95. Interactivity without JavaScript.                  |
| <b>&lt;b&gt;JIT compiler&lt;/b&gt;</b>        | Tailwind v3 uses Just-In-Time compilation — generates only the CSS classes you actually use. Tiny    |
| <b>&lt;b&gt;tailwind.config.js&lt;/b&gt;</b>  | Extend the framework: add custom colours, fonts, spacing, or screen breakpoints.                     |
| <b>&lt;b&gt;Purging&lt;/b&gt;</b>             | In production, unused CSS is removed. A Tailwind project with 1000 utility classes ships only the ~1 |

Tailwind handles layout and spacing. `index.css` defines Buildfolio's unique visual identity:

[illegible]



Vite is a **next-generation frontend build tool**. It replaced older tools like Create React App (which used Webpack internally). Vite is dramatically faster in development because of its fundamentally different architecture.

|                                                        |                                                                                                                                                 |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;Native ES Modules&lt;/b&gt;</b>            | In dev, Vite serves files as native browser ES modules — no bundling step. The browser requests only the modules it needs for the current page. |
| <b>&lt;b&gt;HMR (Hot Module Replacement)&lt;/b&gt;</b> | When you edit a file, only that module is updated in the browser — no full page reload. State is preserved.                                     |
| <b>&lt;b&gt;Rollup for production&lt;/b&gt;</b>        | Vite uses Rollup to bundle, tree-shake, and optimise for production. Smaller bundles than Webpack.                                              |
| <b>&lt;b&gt;import.meta.env&lt;/b&gt;</b>              | Vite's way to access environment variables. VITE_ prefix required. VITE_API_URL becomes import.meta.env.VITE_API_URL.                           |
| <b>&lt;b&gt;vite.config.js&lt;/b&gt;</b>               | Configure plugins (React plugin), aliases (@/src), proxy (for dev API calls), and build options.                                                |

```
// vite.config.js
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      // In dev, forward /api/* calls to the local backend (avoids CORS in dev)
      "/api": {
        target: "http://localhost:5000",
        changeOrigin: true
      }
    }
  }
});

// Accessing env variables in React code
const apiUrl = import.meta.env.VITE_API_URL;

// .env.local (for local dev, never committed)
VITE_API_URL=http://localhost:5000
```

#### **O: Why does Vite start so much faster than Create React App?**

A: Create React App uses Webpack, which bundles the entire application before serving it — even in development. For large apps this can take 30+ seconds. Vite doesn't bundle in development at all. It serves source files as native ES modules directly to the browser. The browser requests only the modules it needs for the current page. Cold start is near-instant regardless of project size.

Node.js lets JavaScript run on a server. Express is a minimal web framework on top of Node.js that adds routing, middleware, and request/response handling. Together they power Buildfolio's entire backend API.

## Node.js Core Concepts

|                         |                                                                                                                                                |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Event Loop</b>       | Node.js is single-threaded but non-blocking. Instead of waiting for I/O (DB queries, file reads), it registers callbacks to be executed later. |
| <b>CommonJS modules</b> | Node.js uses require() for imports. const express = require('express'). package.json lists all dependencies.                                   |
| <b>process.env</b>      | Access environment variables. All secrets (DB password, JWT secret) are stored here, never in code.                                            |
| <b>async/await</b>      | Modern way to handle asynchronous operations. Avoids callback hell. Every database query in Build                                              |

## Express.js Core Concepts

|                                          |                                                                                                                   |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;Router&lt;/b&gt;</b>         | express.Router() creates a mini-app with its own routes. Mounted in server.js with app.use('/api/auth', router)   |
| <b>&lt;b&gt;Middleware&lt;/b&gt;</b>     | Functions that run between request arrival and response. app.use(express.json()) parses JSON body                 |
| <b>&lt;b&gt;req / res&lt;/b&gt;</b>      | req = the incoming request (headers, body, params, user). res = the response you send back (json, text, redirect) |
| <b>&lt;b&gt;Error handling&lt;/b&gt;</b> | Express error middleware has signature (err, req, res, next). Catches unhandled errors from route handlers        |
| <b>&lt;b&gt;next()&lt;/b&gt;</b>         | Call next() to pass control to the next middleware. In auth.js: verify token, attach req.user, call next()        |

[illegible]

```
app.get("/health", (req, res) => {  
const dbStatus = mongoose.connection.readyState === 1 ? "connected" : "disconnected";  
res.json({ status: "ok", db: dbStatus, time: new Date() });  
});  
  
// ■■ Global Error Handler  
app.use((err, req, res, next) => {  
console.error(err.stack);  
res.status(500).json({ message: "Internal server error" });  
});  
  
app.listen(process.env.PORT || 5000);
```

### O: What is middleware in Express? Give examples from Buildfolio.

A: Middleware is a function that receives (req, res, next) and runs between the request arriving and the response being sent. `express.json()` parses JSON bodies for all routes. `cors()` adds CORS headers. The custom `auth.js` middleware verifies the JWT token — if valid, it attaches `req.user` and calls `next()` to allow the route handler to run; if invalid, it returns 401 immediately. Every protected route uses this middleware.

### O: What is the difference between `app.use()` and `app.get()/app.post()`?

A: `app.use()` runs for ALL HTTP methods and can match path prefixes (`app.use('/api', router)` matches `/api/anything`). `app.get()` and `app.post()` are method-specific route handlers. `app.use()` is typically for middleware (logging, auth, parsing); method-specific handlers are for actual route logic.

### O: Why is Node.js good for API servers despite being single-threaded?

A: Most API server work is I/O-bound — waiting for database queries, external API calls, file reads. Node.js uses an event loop: instead of blocking a thread while waiting for I/O, it registers a callback and handles other requests. When the I/O completes, the callback runs. This allows Node.js to handle thousands of concurrent connections with a single thread — very efficient for Buildfolio's use case.



MongoDB is a **NoSQL document database** that stores data as flexible JSON-like documents. MongoDB Atlas is the cloud-hosted managed service. Mongoose adds structure, validation, and an ORM-like API on top of the raw MongoDB driver.

## MongoDB vs SQL — Key Differences

|                                   |                                                                                                                                   |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>SQL (relational)</b>           | Rigid tables, fixed columns, foreign keys, JOINS. Schema defined upfront. Great for structured data.                              |
| <b>MongoDB (document)</b>         | Flexible documents (JSON/BSON). Each document can have different fields. Nested arrays and objects.                               |
| <b>Why MongoDB for Buildfolio</b> | Portfolio data has variable-length arrays (experiences, projects) and optional nested objects. A SQL database would be too rigid. |
| <b>Collection</b>                 | MongoDB's equivalent of a SQL table. Buildfolio: users collection + portfolios collection.                                        |
| <b>ObjectId</b>                   | Auto-generated 12-byte unique identifier for each document. Used to link portfolios to users.                                     |
| <b>Upsert</b>                     | <code>findOneAndUpdate</code> with <code>upsert:true</code> — creates the document if it doesn't exist, updates it if it does.    |

## Mongoose Schema & Model

```
// models/Portfolio.js
const mongoose = require("mongoose");

const portfolioSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User", // reference to User collection
    required: true,
    unique: true // one portfolio per user
  },
  personalInfo: {
    name: String, title: String, summary: String,
    email: String, phone: String, location: String,
    avatar: String // base64 encoded image string
  },
  experiences: [{ // array of subdocuments
    company: String,
    role: String,
    startDate: String,
    endDate: String,
    location: String,
    description: String
  }],
  skills: {
    technical: [String], // array of strings: ["React", "Node.js"]
    languages: [String],
    tools: [String],
    soft: [String]
  },
  projects: [{
    name: String,
    description: String,
    techStack: String,
```

[illegible]

### O: Why did you choose MongoDB over a relational database like PostgreSQL?

A: Portfolio data is naturally document-shaped. A user's portfolio has an array of work experiences, an array of projects, an object of social links — all nested together. In PostgreSQL, I'd need separate tables for portfolios, experiences, projects, and skills, then JOIN them on every read. In MongoDB, all of that lives in one document — one query retrieves everything. The schema is also flexible, so adding a new field like 'certifications' doesn't require an ALTER TABLE migration.

## Q: What is a Mongoose pre-save hook? Where does Buildfolio use it?

A: A pre-save hook runs automatically before a document is saved to the database. Buildfolio uses it in User.js to hash the password: `userSchema.pre('save', async function(next) { if (this.isModified('password')) { this.password = await bcrypt.hash(this.password, 12); } next(); })`. This means the password is always hashed before storage, regardless of which route creates or updates the user.

### Q: What does upsert mean and where is it used?

A: Upsert is a combination of update + insert. `findOneAndUpdate` with `upsert:true` means: find a document matching the filter; if found, update it; if not found, create it. Buildfolio uses this for saving portfolios — the first time a user saves, there's no portfolio document yet, so it creates one. Every subsequent save updates the existing one. One operation handles both cases.

JWT (JSON Web Token) is an open standard (RFC 7519) for securely transmitting information as a signed JSON object. Buildfolio uses JWT to implement **stateless authentication** — the server never needs to store session data.

## How a JWT Works

|                                        |                                                                                                         |
|----------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;Structure&lt;/b&gt;</b>    | Three base64url-encoded parts joined by dots: header.payload.signature                                  |
| <b>&lt;b&gt;Header&lt;/b&gt;</b>       | {"alg":"HS256","typ":"JWT"} — algorithm and type                                                        |
| <b>&lt;b&gt;Payload&lt;/b&gt;</b>      | {"id":"user123","iat":1700000000,"exp":1700604800} — the data (claims)                                  |
| <b>&lt;b&gt;Signature&lt;/b&gt;</b>    | HMAC-SHA256(base64(header) + '.' + base64(payload), JWT_SECRET). Tamper-proof.                          |
| <b>&lt;b&gt;Verification&lt;/b&gt;</b> | Server re-computes the signature. If it matches, the token is valid. If payload was modified, signature |
| <b>&lt;b&gt;Stateless&lt;/b&gt;</b>    | Server doesn't store sessions. The token itself contains the user ID. No DB lookup needed to verify -   |
| <b>&lt;b&gt;7-day expiry&lt;/b&gt;</b> | exp claim set to now + 7 days. After expiry, jwt.verify() throws an error, returning 401.               |

[illegible]

```
// Axios request interceptor – attaches to every request:  
config.headers.Authorization = `Bearer ${localStorage.getItem("pb_token")}`;
```

**O: What is the difference between stateless (JWT) and stateful (session) authentication?**

A: Session-based auth stores a session ID in a database. On every request, the server queries the DB to check if the session is valid — this requires DB I/O and won't scale horizontally without a shared session store. JWT auth is stateless — the server verifies the token cryptographically using the JWT\_SECRET. No DB lookup needed. This scales infinitely because any server instance can verify the token with just the secret key. The trade-off is you can't instantly revoke a JWT before it expires.

**O: Why is the JWT secret stored in an environment variable?**

A: The JWT\_SECRET is the cryptographic key used to sign and verify all tokens. If an attacker knows this key, they can forge valid tokens for any user ID and bypass authentication entirely. Storing it in process.env means it's never in the codebase or version control. On Render, it's set in the dashboard and injected at runtime.

**O: Can you decode a JWT without the secret?**

A: Yes — the header and payload are base64url encoded, not encrypted. Anyone can decode them. This is why you should never put sensitive data (passwords, credit cards) in the JWT payload. The signature is what provides security — you can read the payload but you can't forge a valid signature without the secret.

**O: How does Buildfolio handle token expiry?**

A: When a JWT expires, jwt.verify() throws a TokenExpiredError, which the auth middleware catches and returns a 401. The Axios response interceptor on the frontend detects the 401 status, clears localStorage (removing the expired token and user object), and redirects to the home page. The user must log in again to get a fresh 7-day token.

bcryptjs implements the bcrypt password hashing algorithm in pure JavaScript. Passwords are **never stored in plain text**. Hashing is a one-way function — you can verify a password against a hash but cannot reverse a hash to get the original password.

|                              |                                                                                                                        |
|------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Hashing vs Encryption</b> | Encryption is two-way (you can decrypt). Hashing is one-way. Correct approach for passwords is always hashing.         |
| <b>Salt</b>                  | A random string added to the password before hashing. Each user gets a unique salt, stored in the hash.                |
| <b>Cost factor (rounds)</b>  | bcrypt runs $2^{\text{rounds}}$ iterations. Cost 12 = 4096 iterations. Higher = slower hash = harder to brute-force.   |
| <b>Adaptive</b>              | As hardware gets faster, increase the cost factor. bcrypt slows down attackers proportionally.                         |
| <b>Hash format</b>           | <code>\$2a\$12\$[22-char-salt][31-char-hash]</code> — the salt is embedded in the hash string itself.                  |
| <b>comparePassword()</b>     | bcrypt.compare() hashes the input with the same salt (extracted from stored hash) and compares. Returns true or false. |

```
// Hashing on registration (via pre-save hook in User.js)
const bcrypt = require("bcryptjs");

// Hash the password before saving
const hashed = await bcrypt.hash("userPassword123", 12);
// Result: "$2a$12$abc123randomsalt...hashedvalue..."

// Verify on login
const isMatch = await bcrypt.compare("userPassword123", hashed); // true
const isMatch = await bcrypt.compare("wrongpassword", hashed); // false

// User.js pre-save hook — hashes automatically before any save
userSchema.pre("save", async function(next) {
  if (!this.isModified("password")) return next(); // skip if password unchanged
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// User.js instance method — used in login route
userSchema.methods.comparePassword = async function(candidate) {
  return bcrypt.compare(candidate, this.password);
};

// routes/auth.js — login
const user = await User.findOne({ email });
const valid = await user.comparePassword(req.body.password);
if (!valid) return res.status(401).json({ message: "Invalid credentials" });
```

#### Q: Why use bcrypt instead of SHA-256 or MD5 for password hashing?

A: MD5 and SHA-256 are general-purpose fast hash functions — they can hash billions of passwords per second on modern GPUs. Bcrypt is deliberately slow (adjustable cost factor) — it can only hash a few hundred per second. A stolen database of bcrypt hashes is practically impossible to brute-force, while MD5 hashes can be cracked in minutes. Bcrypt also automatically salts each hash, preventing precomputed rainbow table attacks.

#### Q: If bcrypt is slow, doesn't that make login slow for users?

A: Bcrypt at cost 12 takes roughly 200-400ms per hash — imperceptible to a user logging in once. It's only a problem for attackers trying to test millions of passwords. This is the intentional trade-off: slow enough to deter

brute-force attacks, fast enough not to affect user experience.

## AXIOS — HTTP CLIENT + INTERCEPTORS

Axios is a promise-based HTTP client. Every piece of data that moves between Buildfolio's frontend and backend goes through the single Axios instance configured in `utils/api.js`.

|                                                      |                                                                                                                                            |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>&lt;code&gt;axios.create()&lt;/code&gt;</b>       | Creates a configured instance with base URL, timeout, and default headers. One instance shared across the app.                             |
| <b>&lt;code&gt;Request interceptor&lt;/code&gt;</b>  | Runs before every request. Buildfolio uses it to attach the JWT: Authorization: Bearer <token>.                                            |
| <b>&lt;code&gt;Response interceptor&lt;/code&gt;</b> | Runs after every response. Buildfolio uses it to catch 401s and auto-logout the user.                                                      |
| <b>&lt;code&gt;async/await&lt;/code&gt;</b>          | <code>const { data } = await api.get('/api/portfolio')</code> — clean, readable async code.                                                |
| <b>&lt;code&gt;Error handling&lt;/code&gt;</b>       | Axios throws on 4xx/5xx responses. Wrap in try/catch. <code>err.response.data</code> has the server's error message.                       |
| <b>&lt;code&gt;Axios vs fetch()&lt;/code&gt;</b>     | <code>fetch()</code> doesn't throw on 4xx/5xx, needs manual <code>response.json()</code> , and doesn't have interceptors. Axios is better. |

```
// utils/api.js — the single Axios instance
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "https://buildfolio-t1vl.onrender.com",
  timeout: 30000
});

// REQUEST interceptor — attach JWT to every outgoing request
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("pb_token");
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
  },
  (error) => Promise.reject(error)
);

// RESPONSE interceptor — handle 401 globally
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("pb_token");
      localStorage.removeItem("pb_user");
      window.location.href = "/";
    }
    return Promise.reject(error);
  }
);

export default api;
```

```
// ■■ Usage examples ████████████████████████████████████████
```

```
// GET portfolio data
const { data } = await api.get("/api/portfolio");

// PUT (save) portfolio
await api.put("/api/portfolio", portfolioData);
```

```
// POST (AI analysis)
const { data: result } = await api.post("/api/gemini/optimize-resume", {
  portfolioData,
  jobDescription
});

// Error handling
try {
  await api.put("/api/portfolio", portfolioData);
} catch (err) {
  console.error(err.response?.data?.message || "Save failed");
}
```

**O: Why use a single Axios instance instead of calling `axios.get()` directly everywhere?**

A: A shared instance configured once means: the base URL is set in one place (easy to switch between dev and prod), the JWT is attached automatically (no risk of forgetting it on one call), and the 401 logout logic fires for every request without any component needing to handle it. If I called `axios.get()` directly, every API call would need to manually add the Authorization header and handle auth errors — 20+ places to maintain.

**O: What is the difference between request and response interceptors?**

A: Request interceptors run before the request is sent — used to modify the request (add headers, log it). Response interceptors run after the response is received — used to transform the response or handle specific status codes. Buildfolio uses a request interceptor to attach the JWT and a response interceptor to catch 401 responses and log out the user.



React Context provides a way to share state across the component tree without passing props at every level. Buildfolio uses two contexts: **AuthContext** (who is logged in) and **PortfolioContext** (the portfolio data being built).

|                                      |                                                                                                  |
|--------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>&lt;createContext()&lt;/b&gt;</b> | Creates a Context object. Takes a default value (used if a component has no Provider above it).  |
| <b>&lt;Provider&lt;/b&gt;</b>        | A component that wraps your app and makes the context value available to all children.           |
| <b>&lt;useContext()&lt;/b&gt;</b>    | Hook to consume a context value inside any functional component inside the Provider.             |
| <b>&lt;Prop drilling&lt;/b&gt;</b>   | The problem Context solves: passing data through every intermediate component that doesn't need  |
| <b>&lt;Re-render scope&lt;/b&gt;</b> | Only components that consume the context re-render when its value changes. Others are unaffected |
| <b>&lt;Custom hook&lt;/b&gt;</b>     | Wrap useContext in a custom hook (useAuth, usePortfolio) for better DX and error handling.       |

```
// context/PortfolioContext.jsx
import { createContext, useContext, useState, useCallback } from "react";
import api from "../utils/api";

const PortfolioContext = createContext(null);

export function PortfolioProvider({ children }) {
  const [portfolioData, setPortfolioData] = useState({
    personalInfo: {},
    experiences: [],
    education: [],
    skills: { technical:[], languages:[], tools:[], soft:[] },
    projects: [],
    socialLinks: {},
    achievements: ""
  });

  const syncToServer = useCallback(async () => {
    await api.put("/api/portfolio", portfolioData);
  }, [portfolioData]);

  const loadFromServer = useCallback(async () => {
    const { data } = await api.get("/api/portfolio");
    setPortfolioData(data);
  }, []);

  return (
    <PortfolioContext.Provider value={{ portfolioData, setPortfolioData, syncToServer, loadFromServer }}>
      {children}
    </PortfolioContext.Provider>
  );
}

// Custom hook with guard clause
export const usePortfolio = () => {
  const ctx = useContext(PortfolioContext);
  if (!ctx) throw new Error("usePortfolio must be used within PortfolioProvider");
  return ctx;
};
```

```
// App.jsx – wrap everything in providers
function App() {
  return (
    <AuthProvider>
    <PortfolioProvider>
    <BrowserRouter>...</BrowserRouter>
    </PortfolioProvider>
    </AuthProvider>
  );
}

// Usage anywhere in the component tree
function PreviewPage() {
  const { portfolioData, syncToServer } = usePortfolio();
  const { user } = useAuth();
  // No props needed – context provides everything
}
```

### **O: When would you use Redux instead of Context API?**

A: Context is fine for Buildfolio's scale — 2 global states, updated infrequently. Redux becomes worth the complexity when you have many pieces of state updated from many places, need time-travel debugging, need middleware for side effects (Redux Thunk/Saga), or have complex derived state across many reducers. Context re-renders all consumers when the value changes — in large apps with many context consumers this can cause performance issues that Redux with selective subscriptions would avoid.

### **O: What is prop drilling and why is it a problem?**

A: Prop drilling is passing data as props through multiple intermediate components that don't need the data themselves — only their children do. For example: App -> Dashboard -> Sidebar -> UserAvatar all need the user object, but only UserAvatar displays it. With Context, UserAvatar calls useAuth() directly. Prop drilling makes components fragile — adding a new layer means threading props through another component that doesn't care about them.

Buildfolio uses Google's Gemini 1.5 Flash model for ATS resume optimisation. Users paste a job description; the backend sends a structured prompt to Gemini and returns an ATS score, keyword analysis, and an optimised resume summary.

## LLM / Generative AI Concepts

|                                              |                                                                                                                        |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;LLM&lt;/b&gt;</b>                | Large Language Model. Trained on vast text data to predict likely next tokens. Generates coherent, creative responses. |
| <b>&lt;b&gt;Prompt Engineering&lt;/b&gt;</b> | Crafting inputs that guide the model to produce specific, reliable output formats. Critical for production use.        |
| <b>&lt;b&gt;Temperature&lt;/b&gt;</b>        | Controls randomness. 0 = deterministic/focused (best for structured tasks). 1 = creative/varied.                       |
| <b>&lt;b&gt;Max tokens&lt;/b&gt;</b>         | Limits response length. Prevents excessively long (costly) responses.                                                  |
| <b>&lt;b&gt;JSON mode&lt;/b&gt;</b>          | Prompting the model to respond only in valid JSON. Enables programmatic parsing of AI output.                          |
| <b>&lt;b&gt;Hallucination&lt;/b&gt;</b>      | LLMs can generate plausible-sounding but false information. For Buildfolio this is low risk — it's analysed by humans. |
| <b>&lt;b&gt;Gemini 1.5 Flash&lt;/b&gt;</b>   | Google's fast, efficient model optimised for structured tasks. Free tier available via AI Studio.                      |

```
// routes/gemini.js — ATS optimization
router.post("/optimize-resume", auth, async (req, res) => {
  const { portfolioData, jobDescription } = req.body;

  const prompt = `
  You are an expert ATS (Applicant Tracking System) analyst and resume coach.
  Analyse this resume data against the job description below.

  RESUME DATA:
  ${JSON.stringify(portfolioData, null, 2)}

  JOB DESCRIPTION:
  ${jobDescription}

  IMPORTANT: Respond ONLY with a valid JSON object. No markdown, no backticks, no explanation.
  {
    "atsScore": <integer 0-100>,
    "matchedKeywords": ["keyword1", "keyword2"],
    "missingKeywords": ["keyword3", "keyword4"],
    "optimizedSummary": "<rewritten summary optimized for this JD>",
    "reorderedSkills": {
      "technical": ["most relevant first..."],
      "languages": ["most relevant first..."]
    },
    "experienceEnhancements": [
      { "role": "Job Title at Company", "suggestions": ["bullet point", "bullet point"] }
    ],
    "scoreBreakdown": {
      "keywordMatch": 75,
      "experienceMatch": 80,
      "skillsMatch": 70,
      "formatting": 90
    }
  }
  `;
}
```

```
// Call Gemini REST API directly (no SDK needed)
const response = await fetch(
  `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${process.env.GEMINI_API_KEY}`,
  {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      contents: [{ parts: [{ text: prompt }] }],
      generationConfig: {
        temperature: 0.3, // low = focused output
        maxOutputTokens: 2048
      }
    })
  }
);

const data = await response.json();
const rawText = data.candidates[0].content.parts[0].text;
// Strip any accidental markdown code fences
const cleaned = rawText.replace(/```json|```/g, "").trim();
const result = JSON.parse(cleaned);
res.json(result);
});
```

#### **O: What is prompt engineering and why does it matter?**

A: Prompt engineering is the practice of crafting model inputs to reliably produce specific outputs. In Buildfolio's ATS optimizer, the prompt specifies the model's role ('ATS expert'), provides structured data, defines the exact JSON schema for the response, and explicitly says 'no markdown, no backticks.' Without these instructions, the model might wrap the JSON in code fences, add explanatory text, or use a different field structure — all of which would break the `JSON.parse()` call.

#### **O: Why is the Gemini API called from the backend, not the frontend?**

A: The Gemini API key must stay secret. If called from the frontend, the API key would be visible in the browser's network tab and in the bundled JavaScript — anyone could extract it and use my quota. Calling it from the backend keeps the key server-side as an environment variable. The frontend only communicates with our own authenticated backend endpoint.

#### **O: How do you handle cases where the AI doesn't return valid JSON?**

A: The response is sanitised with `replace(/```json|```/g, "").trim()` to strip markdown code fences before `JSON.parse()`. The route is also wrapped in a `try/catch` — if `JSON.parse` fails, the route returns a 500 with an error message. For production robustness, you could also use a retry strategy or set the model's `response_mime_type` to `application/json` to enforce JSON output at the API level.

The export feature generates HTML, CSS, and JS files for the user's chosen template, packages them into a ZIP, and streams it directly to the browser as a file download — all without writing anything to disk.

|                                               |                                                                                                      |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;archiver&lt;/b&gt;</b>            | Node.js library for creating ZIP and TAR archives. Supports streaming output.                        |
| <b>&lt;b&gt;Streaming&lt;/b&gt;</b>           | Archive is piped directly to the HTTP response stream. No temp file created. Memory-efficient.       |
| <b>&lt;b&gt;archive.pipe()&lt;/b&gt;</b>      | Connects the archive output stream to the response stream. Data flows: archiver -> HTTP response     |
| <b>&lt;b&gt;archive.append()&lt;/b&gt;</b>    | Adds a string, buffer, or file stream to the archive with a given filename.                          |
| <b>&lt;b&gt;archive.finalize()&lt;/b&gt;</b>  | Signals that all files have been added. Flushes remaining data and ends the stream.                  |
| <b>&lt;b&gt;Content-Disposition&lt;/b&gt;</b> | HTTP header telling the browser to download the response as a file: attachment; filename=portfolio.. |

[illegible]

### Q: Why stream the ZIP directly to the response instead of saving it to a file first?

A: Writing to disk requires: finding a temp directory, generating a unique filename, writing the file, reading it back, streaming it, then deleting the temp file. Streaming directly skips all of that. The ZIP is generated and sent in one pass with no disk I/O. On cloud servers like Render with ephemeral storage and limited disk space, this is particularly important. It's also faster and uses less memory since data is never held in memory all at once.

Nodemailer handles the forgot-password email flow. A user requests a reset, the backend generates a time-limited token, saves it, and emails a link. The link contains the token — clicking it opens a reset form.

|                                              |                                                                                                                           |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;SMTP&lt;/b&gt;</b>               | Simple Mail Transfer Protocol. Standard for sending email. Nodemailer sends emails via an SMTP server.                    |
| <b>&lt;b&gt;Transporter&lt;/b&gt;</b>        | A configured Nodemailer object: SMTP host, port, credentials. Like an email client config.                                |
| <b>&lt;b&gt;Gmail App Password&lt;/b&gt;</b> | A 16-char password generated specifically for app use. Your real Gmail password is NOT used.                              |
| <b>&lt;b&gt;Reset token&lt;/b&gt;</b>        | <code>crypto.randomBytes(32).toString('hex')</code> — 32 random bytes → 64 hex chars. Cryptographically secure.           |
| <b>&lt;b&gt;1-hour expiry&lt;/b&gt;</b>      | Token stored with <code>Date.now() + 3600000</code> expiry. Query checks <code>resetExpiry: { \$gt: Date.now() }</code> . |
| <b>&lt;b&gt;Single-use token&lt;/b&gt;</b>   | After reset, <code>resetToken</code> and <code>resetExpiry</code> are set to null — the token can't be reused.            |

```
// routes/auth.js – forgot + reset password
const nodemailer = require("nodemailer");
const crypto = require("crypto");

const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST, // smtp.gmail.com
  port: parseInt(process.env.SMTP_PORT), // 587
  secure: false,
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS }
});

router.post("/forgot-password", async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user) return res.status(404).json({ message: "No account with that email" });

  const token = crypto.randomBytes(32).toString("hex");
  const expiry = Date.now() + 60 * 60 * 1000; // 1 hour

  user.resetToken = token;
  user.resetExpiry = expiry;
  await user.save({ validateBeforeSave: false });

  const url = `${process.env.FRONTEND_URL}/reset-password?token=${token}`;

  await transporter.sendMail({
    from: `"Buildfolio" <${process.env.SMTP_USER}>`,
    to: user.email,
    subject: "Reset your Buildfolio password",
    html: `<h2>Password Reset</h2>
    <p><a href="${url}">Click here to reset your password</a></p>
    <p>This link expires in 1 hour.</p>`
  });

  res.json({ message: "Reset email sent" });
});

router.post("/reset-password", async (req, res) => {
  const user = await User.findOne({
    resetToken: req.body.token,
    resetExpiry: { $gt: Date.now() } // must not be expired
  });
  if (!user) return res.status(404).json({ message: "Reset link expired" });

  user.resetToken = null;
  user.resetExpiry = null;
  await user.save({ validateBeforeSave: false });

  res.json({ message: "Password reset successful" });
});
```

```
});

if (!user) return res.status(400).json({ message: "Invalid or expired link" });

user.password = req.body.newPassword; // pre-save hook hashes this
user.resetToken = null; // invalidate – single use
user.resetExpiry = null;
await user.save();

res.json({ message: "Password reset successful" });
});
```



## Vercel — Frontend

|                                         |                                                                                          |
|-----------------------------------------|------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;What it does&lt;/b&gt;</b>  | Hosts the React/Vite frontend. Auto-deploys from GitHub on every push to main.           |
| <b>&lt;b&gt;CDN&lt;/b&gt;</b>           | Files served from 100+ edge locations worldwide. Fast load times globally.               |
| <b>&lt;b&gt;Build command&lt;/b&gt;</b> | vite build — produces /dist folder with optimised, bundled static assets.                |
| <b>&lt;b&gt;SPA routing&lt;/b&gt;</b>   | vercel.json rewrite: all paths -> index.html. React Router handles the rest client-side. |
| <b>&lt;b&gt;Env variables&lt;/b&gt;</b> | VITE_API_URL set in Vercel dashboard — injected at build time into the JS bundle.        |
| <b>&lt;b&gt;Free tier&lt;/b&gt;</b>     | Unlimited personal projects, custom domains, HTTPS, 100GB bandwidth — free.              |

## Render — Backend

|                                            |                                                                                                       |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;What it does&lt;/b&gt;</b>     | Hosts the Node.js/Express backend. Auto-deploys from GitHub.                                          |
| <b>&lt;b&gt;Build command&lt;/b&gt;</b>    | npm install — installs all dependencies from package.json.                                            |
| <b>&lt;b&gt;Start command&lt;/b&gt;</b>    | npm start (node server.js) — starts the Express server.                                               |
| <b>&lt;b&gt;render.yaml&lt;/b&gt;</b>      | Declarative deployment config: build/start commands, env var references, instance type.               |
| <b>&lt;b&gt;Free tier caveat&lt;/b&gt;</b> | Free services spin down after 15 min of inactivity. First request after idle takes ~30s (cold start). |
| <b>&lt;b&gt;Env variables&lt;/b&gt;</b>    | MONGO_URI, JWT_SECRET, GEMINI_API_KEY set in Render dashboard — never in code.                        |

## MongoDB Atlas — Database

|                                        |                                                                                                         |
|----------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;What it does&lt;/b&gt;</b> | Cloud-hosted MongoDB. No server to manage. Automatic backups, monitoring, and scaling.                  |
| <b>&lt;b&gt;Free tier&lt;/b&gt;</b>    | M0 cluster: 512MB storage. Sufficient for hundreds of portfolio records.                                |
| <b>&lt;b&gt;Connection&lt;/b&gt;</b>   | MONGO_URI connection string in Render env var. Mongoose connects on server startup.                     |
| <b>&lt;b&gt;IP whitelist&lt;/b&gt;</b> | Atlas can restrict which IPs can connect. For cloud deployments, allow 0.0.0.0/0 (all IPs) since Render |

```
# render.yaml
services:
- type: web
  name: buildfolio-api
  env: node
  rootDir: backend
  buildCommand: npm install
  startCommand: npm start
  envVars:
  - key: MONGO_URI
  sync: false # manually set in Render dashboard
  - key: JWT_SECRET
  sync: false
  - key: GEMINI_API_KEY
  sync: false
  - key: FRONTEND_URL
  value: https://buildfolio-one.vercel.app
```

**O: What is a CDN and why does Vercel use one?**

A: A Content Delivery Network is a globally distributed network of servers that cache and serve static files. When a user in Mumbai requests Buildfolio, Vercel serves the HTML/JS/CSS from a nearby Mumbai edge node rather than from a single origin server in the US. This reduces latency and makes the app load faster everywhere in the world.

**O: What happens in the CI/CD pipeline when you push code to GitHub?**

A: Pushing to main triggers both Vercel and Render webhooks simultaneously. Vercel runs vite build, generating an optimised /dist bundle, then deploys it to its global CDN — usually in under 30 seconds. Render runs npm install then restarts the Node.js server with the new code. Both are automatic — no manual deployment steps. If either build fails, the previous version stays live.

Security in Buildfolio is layered — each layer addresses a specific class of attack.

|                                               |                                                                                                        |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;CORS&lt;/b&gt;</b>                | Restricts which origins can make cross-origin requests to the backend. Only the Vercel domain is wh    |
| <b>&lt;b&gt;Password hashing&lt;/b&gt;</b>    | bcryptjs with cost 12. Passwords never stored as plaintext. A stolen database is useless without the   |
| <b>&lt;b&gt;JWT expiry&lt;/b&gt;</b>          | 7-day token expiry limits the window of a stolen token. 1-hour password reset token expiry limits phis |
| <b>&lt;b&gt;Env variables&lt;/b&gt;</b>       | All secrets (DB password, JWT secret, API keys) in process.env. Never in source code or version co     |
| <b>&lt;b&gt;Auth middleware&lt;/b&gt;</b>     | Every portfolio, export, and AI route requires a valid JWT. No public access to user data.             |
| <b>&lt;b&gt;HTTPS&lt;/b&gt;</b>               | Vercel and Render provide free TLS certificates. All traffic is encrypted in transit.                  |
| <b>&lt;b&gt;Single-use tokens&lt;/b&gt;</b>   | Password reset tokens are nullified after use. Cannot be replayed.                                     |
| <b>&lt;b&gt;Minimal JWT payload&lt;/b&gt;</b> | Only user._id is stored in the JWT. No sensitive data (email, name) — those require a DB lookup.       |

### What Could Be Improved (honest assessment)

- **Rate limiting** — express-rate-limit on auth routes to prevent brute-force login attempts
- **Input validation** — express-validator to sanitise all user inputs against XSS and injection
- **Refresh tokens** — short-lived access tokens (15min) with long-lived refresh tokens for better security
- **HTTPS-only cookies** — storing JWT in httpOnly cookies instead of localStorage prevents XSS token theft
- **MongoDB injection** — Mongoose's schema typing provides some protection; \$where queries should be avoided

#### O: What are the security risks of storing the JWT in localStorage?

A: localStorage is accessible to any JavaScript on the page. If the site has an XSS vulnerability (malicious script injected via user input), the script could read the token from localStorage and send it to an attacker. The more secure approach is storing the JWT in an httpOnly cookie — httpOnly cookies can't be read by JavaScript, only sent automatically with requests. The trade-off is more complex CSRF protection. For Buildfolio's current scale, localStorage is an acceptable pragmatic choice.

#### O: How does CORS protect Buildfolio?

A: CORS is enforced by the browser. When the frontend at buildfolio-one.vercel.app makes a request to the backend, the browser sends an Origin header. The backend's CORS middleware checks if the origin matches FRONTEND\_URL. If it doesn't, the browser blocks the response. This prevents other websites from making authenticated requests to Buildfolio's API on behalf of a logged-in user — a CSRF-like attack. Note: CORS is a browser mechanism — it doesn't protect against direct server-to-server requests or curl.

Organised by topic. Every answer is written as a confident, specific response referencing Buildfolio. Study these until you can answer them without reading.

## React & Frontend

### **Q: Explain React's component lifecycle.**

A: In functional components, lifecycle phases are handled by hooks. Mounting = `useEffect` with `[]`. Updating = `useEffect` with dependencies. Unmounting = return a cleanup function from `useEffect`. In Buildfolio, `BuilderPage` mounts and calls `loadFromServer()` once. If the user's authentication state changes, a separate `useEffect` responds to that dependency.

### **Q: What is the difference between `useMemo` and `useCallback`?**

A: `useMemo` memoises the return value of a function — use when computing an expensive derived value. `useCallback` memoises the function reference itself — use when passing a callback to a child component that's wrapped in `React.memo`, to prevent unnecessary re-renders. In Buildfolio, `syncToServer` and `loadFromServer` in `PortfolioContext` could use `useCallback` to stabilise their references.

### **Q: How does React re-rendering work?**

A: A component re-renders when its state changes, its props change, or its parent re-renders. React then runs the component function again, produces a new virtual DOM, diffs it against the previous one (reconciliation), and patches only the changed DOM nodes. React 19's compiler automatically memoises components to prevent unnecessary re-renders.

### **Q: What is code splitting and does Buildfolio use it?**

A: Code splitting means breaking the JS bundle into smaller chunks loaded on demand instead of upfront. `React.lazy()` and `Suspense` enable component-level code splitting. Vite handles automatic chunk splitting for routes. Buildfolio could implement route-based code splitting by lazy-loading each page — the template files in particular are large and good candidates.

### **Q: Why is the `key` prop important in lists?**

A: React uses keys to identify which items changed, were added, or removed when re-rendering arrays. Without keys, React re-renders the entire list on any change. Keys must be unique and stable — using array index as key is problematic because inserting at the beginning shifts all indices, causing React to re-render every item.

## Node.js, Express & Backend

### **Q: What is the event loop in Node.js?**

A: Node.js runs on a single thread with an event loop. When an async operation (DB query, HTTP call) is initiated, Node.js doesn't block — it registers a callback and moves to the next request. When the I/O completes, the callback is placed in the event queue and executed on the next loop iteration. This model handles thousands of concurrent connections without thread overhead.

### **Q: What is the difference between `app.use()` and router-level middleware?**

A: `app.use()` applies middleware globally to all routes in the entire Express app. Router-level middleware (`router.use()`) applies only to routes defined in that router. In Buildfolio, CORS and JSON parsing are global (`app.use`). The JWT auth check is router-level — applied at the top of `portfolio.js`, `export.js`, and `gemini.js` routers,

protecting all routes in those files.

**O: How would you add rate limiting to the auth routes?**

A: Install express-rate-limit. Create a limiter: `const limiter = rateLimit({ windowMs: 15*60*1000, max: 10 })`. Apply to auth routes: `router.use('/login', limiter)`. This limits each IP to 10 login attempts per 15 minutes, preventing brute-force attacks.

**O: What HTTP status codes does Buildfolio use and why?**

A: 200 OK for successful GETs and PUTs. 201 Created for successful registration. 400 Bad Request for invalid input (email taken, invalid token). 401 Unauthorized for missing or invalid JWT. 404 Not Found when a resource doesn't exist. 500 Internal Server Error for unexpected exceptions.

**O: Explain `asvnc/await` vs Promises.**

A: Promises represent an eventual value. `.then()/.catch()` handle resolution. `async/await` is syntactic sugar on top — an async function returns a Promise; `await` pauses execution until the Promise resolves. `await` makes async code look synchronous and easier to read. All Buildfolio's route handlers are async functions using `await` for MongoDB operations.

## Database & MongoDB

### O: What is an index in MongoDB and when would you add one?

A: An index is a data structure that speeds up queries on a specific field. Without an index, MongoDB scans every document (collection scan). Buildfolio should have indexes on: User.email (unique index — used in login findOne), Portfolio.user (used in every portfolio query). Mongoose unique: true automatically creates a unique index.

### O: What is the difference between findOne and find?

A: findOne returns the first matching document as an object, or null if none found. find returns a cursor of all matching documents (use .toArray() or iterate). In Buildfolio, findOne is used for login (find user by email) and portfolio retrieval (find portfolio by user ID) — we expect at most one result.

### O: What is the \$gt operator in MongoDB?

A: \$gt means 'greater than'. It's a comparison query operator. In Buildfolio's password reset: { resetExpiry: { \$gt: Date.now() } } means 'find users where resetExpiry is greater than the current timestamp' — i.e., the token hasn't expired yet. Other operators: \$lt, \$gte, \$lte, \$ne, \$in, \$exists.

### O: How does Mongoose schema validation work?

A: Mongoose validates documents against the schema before saving to the database. required: true throws a ValidationError if the field is missing. type: String coerces or rejects non-strings. unique: true creates a DB index (not a schema validator — use mongoose-unique-validator for better error messages). Custom validators can use validate: { validator: fn, message: 'Error' }.

### O: What is the difference between .save() and findOneAndUpdate?

A: .save() updates the entire document and runs all Mongoose middleware (pre/post hooks). findOneAndUpdate() sends a MongoDB update operation directly, bypassing Mongoose middleware. Buildfolio uses .save() when updating the user's password so the pre-save hash hook runs. It uses findOneAndUpdate with upsert for portfolio saves where the hook isn't needed.

## Authentication & Security

### O: What is the difference between authentication and authorisation?

A: Authentication is verifying who you are (login — proving you are the user associated with this email). Authorisation is verifying what you're allowed to do (the JWT middleware confirms you're authenticated; the route checks if you own the resource you're requesting). In Buildfolio, the auth middleware handles authentication; the portfolio routes implicitly authorise by querying { user: req.user.id }.

### O: What would happen if two users had the same JWT secret?

A: If somehow two Buildfolio instances shared the same JWT\_SECRET, a token issued by one would be valid on the other. In a distributed/microservices system this is actually desirable. The problem is if the secret leaks — anyone can forge tokens for any user.

### O: How would you implement JWT refresh tokens?

A: Short-lived access tokens (15min) are sent with every API request. Long-lived refresh tokens (30 days) are stored in httpOnly cookies. When the access token expires (401), the frontend sends the refresh token to a /auth/refresh endpoint. The server verifies it, issues a new access token. This limits the window for stolen access tokens while keeping users logged in.

### Q: What is a rainbow table attack and how does bcrypt prevent it?

A: A rainbow table is a precomputed table of hash->password mappings. An attacker with a stolen database can look up common passwords in the table. Bcrypt prevents this with per-user salts — each password is hashed with a unique random salt, so 'password123' hashes differently for every user. Precomputed rainbow tables are useless because they'd need a separate table for every possible salt.

**Q: Why is the password reset token stored in the database instead of being a JWT?**

A: A JWT can't be revoked before expiry — once issued, it's valid until it expires. A database-stored token can be deleted (or nullified) immediately after use, making it single-use. If we used a JWT as the reset token, a user who clicked the link multiple times (or a replay attacker) could reset the password multiple times within the expiry window.

---

## System Design & Architecture

### O: How would you scale Buildfolio to 100,000 users?

A: Frontend: already globally distributed via Vercel CDN — scales automatically. Backend: move from Render free tier to multiple instances behind a load balancer. Since JWT auth is stateless, any instance can handle any request — horizontal scaling is trivial. Database: MongoDB Atlas scales vertically (larger instance) or horizontally (sharding). Add Redis caching for frequently-read portfolio data. Add a CDN for generated portfolio assets.

### O: What are the trade-offs of your monorepo structure?

A: Advantages: one push deploys both, shared context, one PR per feature. Disadvantages: as the project grows, both platforms rebuild on every push regardless of which folder changed (though both Vercel and Render have path-filtering). A separate frontend and backend repo would give cleaner CI/CD isolation. For Buildfolio's current scale, monorepo is clearly the right choice.

### O: How would you add real-time collaboration (two users editing the same portfolio)?

A: WebSockets via Socket.io. Each user's browser connects to a socket server. Changes from one user are broadcast to all other connected users editing the same portfolio. Operational transforms or CRDTs would resolve conflicts if two users edit the same field simultaneously. MongoDB change streams could trigger socket events when the database is updated.

### O: Why did you separate templates into their own folder with a registry index.js?

A: Each template is self-contained — it only knows how to render data, not how to collect or save it. The registry (index.js) exports all templates with IDs and names. Both TemplatePage and PreviewPage import from this single file — they're always in sync. Adding a new template means creating one folder and adding one entry to the registry. No other files change.

### O: How does Buildfolio handle errors?

A: Frontend: Axios response interceptor handles 401 globally. All API calls are wrapped in try/catch with user-facing error messages. Backend: each route handler has try/catch returning appropriate status codes. A global Express error handler catches any unhandled exceptions. The /health endpoint allows monitoring the DB connection status.

---

## AI & Gemini Integration

### O: What is temperature in an LLM API call?

A: Temperature controls output randomness. At 0, the model always picks the highest-probability next token — output is deterministic and focused. At 1, it samples more broadly — creative but less predictable. Buildfolio uses temperature 0.3 for ATS analysis because we need consistent, structured JSON output, not creative variation.

### O: How would you improve the ATS optimizer's reliability?

A: Set response\_mime\_type: 'application/json' in the Gemini API request to enforce JSON at the API level. Add retry logic with exponential backoff for API failures. Validate the response structure with a schema (Zod or Joi) before returning to frontend. Cache analysis results for the same portfolio+JD combination to reduce API costs.

### O: What are the cost considerations for using Gemini in production?

A: Gemini 1.5 Flash charges per input + output token. Each ATS analysis sends the user's entire portfolio as JSON (~2000 tokens) plus the job description (~500 tokens). At scale, this adds up. Mitigations: only call Gemini when the user explicitly clicks Analyze, implement result caching (same JD + same portfolio = cached result), and set maxOutputTokens to avoid runaway responses.



## The 2-Minute Project Pitch

*"Buildfolio is a full-stack web application I built that lets anyone create a professional portfolio and ATS-optimised resume in minutes. The frontend is React 19 with Vite, TailwindCSS, and a custom glassmorphism design system. The backend is Node.js and Express with JWT authentication and MongoDB Atlas for persistence. The most interesting feature is the AI layer — users paste a job description and Gemini 1.5 Flash analyses their resume, returning an ATS score, matched and missing keywords, and an optimised summary. The app is fully deployed: frontend on Vercel with a global CDN, backend on Render, and database on MongoDB Atlas. Every push to GitHub auto-deploys both."*

## Technical Decisions You Can Defend

### Why React over Vue or Angular?

React has the largest ecosystem, best job market fit, and I was already familiar with it. For a component-heavy app like a multi-step builder, React's composability is a natural fit.

### Why MongoDB over PostgreSQL?

Portfolio data is document-shaped with variable-length nested arrays. MongoDB's flexible schema maps directly to this — no joins, no separate tables for experiences and projects.

### Why JWT over sessions?

Stateless auth scales horizontally without a shared session store. The 7-day token eliminates the need for refresh tokens at this scale while keeping the implementation simple.

### Why Vite over Create React App?

Vite's native ES module dev server starts instantly regardless of project size. CRA bundles everything before serving — Vite serves modules directly to the browser.

### Why Context over Redux?

Buildfolio only has 2 global states updated from a handful of places. Redux would add significant boilerplate with no benefit at this scale.

## Challenges & How You Solved Them

|                                                   |                                                                                                                                                                                               |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>&lt;b&gt;Streaming ZIP download&lt;/b&gt;</b>  | Initial approach wrote files to disk, which failed on Render's ephemeral storage. Solution: use <code>stream</code> and <code>fs.createReadStream</code> to serve files directly from memory. |
| <b>&lt;b&gt;Gemini JSON reliability&lt;/b&gt;</b> | Model occasionally wrapped JSON in markdown fences despite instructions. Solution: strip code fences and parse the inner JSON.                                                                |
| <b>&lt;b&gt;SPA routing on Vercel&lt;/b&gt;</b>   | Direct URL access to <code>/builder</code> returned 404. Solution: <code>vercel.json</code> rewrite rule serves <code>index.html</code> for all <code>/builder/*</code> requests.             |
| <b>&lt;b&gt;Profile photo storage&lt;/b&gt;</b>   | Base64 encoding large images bloated MongoDB documents. Current solution stores base64 in a separate field and uses <code>fs.readFile</code> to serve images.                                 |

## What You'd Do Differently / What's Next

- **Rate limiting** on auth routes to prevent brute-force attacks (`express-rate-limit`)

- **Input validation** with Zod or express-validator to prevent XSS and injection
- **Image storage** on S3 or Cloudinary instead of base64 in MongoDB
- **httpOnly cookies** for JWT storage instead of localStorage (prevents XSS token theft)
- **TypeScript** on both frontend and backend for type safety and better developer experience
- **Testing** — Jest for unit tests, React Testing Library for component tests, Supertest for API tests
- **Real-time preview** — as users type in the builder, the preview updates live using debouncing

---

**You built every layer of this stack. You understand every decision.**

Walk into that interview with confidence.